When creating an alias pointing to multiple indexes, at least one index must have an `is_write_index` property set to `true`. For example, this snippet creates a `cars3` index with `is_write_index` enabled:

```
PUT cars3
{
  "aliases": {
    "cars_alias": {
    "is_write_index":true
    }
  }
}
```

Similarly, we can create an alias pointing to multiple indexes using a wildcard (we must make sure one of the indexes has an `is_write_index` property set to `true`).

Listing 6.15 Creating an alias with a wildcard

```
PUT cars*/_alias/wildcard_cars_alias
```

All indexes prefixed with cars

Once the aliases are created, getting the index (`GET <alias_name>`) details will reflect the aliases defined on the index. `GET cars` returns the index with all aliases created on that index (in addition to all the mappings and settings).

Listing 6.16 Fetching aliases, settings, and mappings for an index

```
GET cars
```

Now that we understand the mechanism of creating aliases, let's see how to fetch the alias details. Similar to settings and mappings, we can send a `GET` request to the `_alias` endpoint to fetch the details of the aliases.

Listing 6.17 Getting an alias on a single index

```
GET my_cars_alias/_alias
```

Of course, we can also extend the same command to multiple aliases.

Listing 6.18 Fetching aliases associated with multiple indexes

```
GET all_cars_alias,my_cars_alias/_alias
```

MIGRATING DATA WITH ZERO DOWNTIME USING ALIASES

Configuration settings for indexes in production may have to be updated with newer properties, perhaps due to a new business requirement or a technical enhancement (or to fix a bug). The new properties may not be compatible with the existing data in that index, in which case we can create an index with new settings and migrate the data from the old index into the brand-new index.